Generative Construction of 6 Geometric Freedoms

Kirill Kovlekov
Independent Researcher

November 2026

A thought experiment on how a binary geometric operand breaches into the complex plane

---

0. The rules of the thought experiment

Let's play a game.

We have a **binary geometric operand**: a minimal combinatorial object with three faces - informatic, geometric, and algebraic - united under a single grammar. For the sake of the experiment, let's call it a **"dot."**

The starting condition of a dot is as follows:

1. **Informatic:** A bit of information with two states (0; 1).
2. **Geometric:** A point that serves as its own coordinates.
3. **Algebraic:** Operated by a minimal grammar of two moves: +1 (flag up) and -1 (flag down).
4. **The Goal:** Reach the complex plane. The dot must grow through incremental algebraic development, every new operation must be reflected in its geometry and informatics.

Essentially, this is a schema-design problem: We treat the dot as a compressed representation of three interfaces that must remain coherent as the algebra expands. To find the minimal representation, we will step through the number systems (N→Z→Q→C), introduce the necessary operations at each stage, and observe how the dot's geometry and informatics must adapt.

---

1. Binary environment (N)

At the start of the thought experiment, we have a dot with a present/absent switch. In other words, a single bit represented as a point. A perfect starting point for our automata.

Flipping the bit doesn't amount to much. But this changes the moment we take the abstraction of the flip and apply it to the existence of the bit itself.

Now we have +1; -1 (bit) added to the moveset of +1; -1 (flag). This self-evidently yields a binary string. Under the rules of the game, we are now one step closer to the complex plane: our dot has transformed into a binary string.

Then we repeat the operation and apply the flip to the string itself: +1 string; -1 string.

This move lets us build a binary environment with three scales of operation:
I) **Object** – bit.
II) **Structure** – string.
III) **Instance** – binary environment.

When we apply the same moveset to the whole binary environment, the construction gains the power to flip the existence of the instance it operates on.

We could try to create more layers of abstraction - binary arrays, for instance - but we are constrained by the minimal grammar (+1; -1 as the moveset; changing it would create another environment).

An array is not a longer string. It is a structure addressed by *two* counts at once: a row and a column. Combining two counts into a single address requires multiplication, and multiplication is not yet in our moveset (unary). The binary environment here is therefore irreducibly one-dimensional. This is the complexity ceiling we can lift ourselves to.

Notice that we have only shifted perspective, not the moveset. Changing the moveset would violate the condition of incremental growth. Therefore, we cannot introduce multiplication here. We are forced to exhaust the +1; -1 *grammar completely.*

Getting back to our construction: what emerges is a fascinating little computational universe, better perceived as a progressive counting: bit flipping produces strings; string flipping produces the whole binary environment; and flipping the binary environment doesn't produce any meaningful distinction with the limitations of our moveset, and as such – flipping it actually voids the whole structure into null.

Given that even after reaching the null we still possess the moveset - we can start over and start flipping bits again – this doesn't mean the termination of our binary geometric operand. Moreover, it gives us the capability to wrap this whole structure by an emergent rule:

Null precedes the bit and supersedes the binary environment.

In practice this means that we have constructed a computational universe that is close to the natural numbers of the ancients Greeks, where null was not a number, but a marked absence.

Why encode this as (N)?

1. An environment of purely discrete objects.
2. We can count bits and strings ad infinitum, each count tied to a "natural" container (a bit or a string).
3. Before we start counting - and once we stop - the default state of the counting instance is null.

As such, we have officially exhausted any meaningful development under this ruleset.

Let's go do integers.

2. Integer environment (Z)

Improving the moveset requires only one move: repeat the operations.

Multiple +1 yields +a. Multiple -1 yields -a. This is the cheapest possible step, and it immediately gives us the informatical operation of this layer:

abstract counting.

"How many flags / strings / instances?" The construction can now answer with a number. This is the move that lets the whole environment undergo a geometric revolution.

First, let's recognize what the binary environment actually was. It was fully discrete. But it had a fatal architectural flaw that terminates it when it tries to operate on its own instance. This fatal flaw can be named – conditional infinity. Conditional as in the natural numbers are transfinite under the condition of continuous counting. But the traditional way of flipping the binary flags is infinitely slower than adding the abstract numbers in bulk with +a operation on integers.

But this doesn't get us anywhere near the complex plane, we can count for infinity and never reach. So, that may be the very answer we are looking for. We can abstract ourselves an infinity and use it in "+a" operation. That way we can harness the infinity to count us out of this dull place.

But the moment that we "invert" the discreteness of our environment – we arrive at a geometry of a line. We all know what a line is: an object that looks a

lot like a string, but is continuous. But a continuous line is truly infinite and we were able to count only discrete objects.

Once the discreteness is inverted, we land on an instance that is a single continuous line bearing no distinction. The promised computational power of infinity isn't accessible because we've traded marked objects for discreteness. We can definitely count, but beyond counting: 1 infinite line – we have nothing to go by.

We can switch it all back off, of course. But that would return us to the binary context with the knowledge that we can count for all eternity and never make a crossing, making it not only a step back from our goal, but also a very futile venture.

The line is infinite. It has no boundary and no internal marks. The counter that marked the binary environment has nothing to mark here. But we can still operate on it.

The moveset +1 and -1 can be applied to themselves to strip off the 1, which stood for the "mark" in a discrete kind of sense. This way our moveset is, in theory, made to match the current state of unmarked existence that is our line.

Speaking geometrically +/- become sort of a unmeasured "gain" operation:

+ stretch

- constrict

- both do not change the properties of the line except some kind of abstract stretching and constricting so this is our next incremental affordance for expanding the grammar. Not that we could measure how it affects the line, so, the tool is up for the task.

Reclaiming discreteness

In the binary environment we already built up to the instance scale. But now our instance is a line with no marks on it – and that's why we have to start from the end:

1. Instance: a continuous line.
The line is infinite. +/- work on it as a knob: stretch or constrict. Nothing is added or subtracted. Only the configuration changes.

2. Structure: still a continuous line.
Turn the constrict knob to its maximum. We can imagine that the constriction reaches a limit condition – a state so constricted that it reaches near to the null, passes through it and begins stretching again. This is a purely geometric

intuition that lets us **invert** a line. Let's mark the point where this happens as the **inversion point**. This allows us to assign direction to a line: negative (-) on the constricting side, positive on the stretching side (+).

3. Object: still a continuous line.
The inversion point is a solid hold we now possess over the line. This means that we can not only count the line (1 line) but also we can recognize a single point on the line (1 inversion point). If we assign any given inversion point to a line that it belongs to – then we can distinguish one line from another!

This is very important because when we assign a mark to the line – it can be asserted as a discrete object. Let's call this discrete object - the **anchor point**.

These three scales are the three interfaces of a singular line:

Anchor point = the line as a point (discrete, countable)

- **Inversion point** = the point where the line inverts; a null (the connection to the binary environment which also has a null)
- **Stretch / Constrict** = the +/- knob on the line (the operations on a line)

And as long as we have an anchor point – we can now string lines together. 1 anchor point – 1 line. 2 anchor points – 2 lines. 3 anchor points – 3 lines. This is the way that the infinite line is tamed into submission and can become the true integer line.

Integer line = an infinite line consisting of infinite lines counted by their anchor points.

We didn't import any discreteness from outside – we have built the discrete integer line by marking it into existence. And with the state of discreteness recovered we can continue the counting journey to the complex plane:

+a / -a = add that many anchor or inversion points.

The broken perfection

Now we expand the moveset once more to speed up the process of counting by adding two more emergent operations to our moveset:

- ×a: add a to itself a times.
- ÷a: partition a units into equal parts.

This is the leap that was forbidden in Section 1. With ×a, the address space opens, and the binary environment could finally become an array. But what did it cost us?

Take an operation of 3÷2. This is a trivial primary school task, but on the perfect mint-fresh integer line it reads: create an inversion point between 3 and 2.

÷a introduces **inversion points that do not sit on integer coordinates**.

Turns out, when we defined the continuous line by its anchor points – we disregarded the fact that inversion points are a different entity that smuggles in the unmarked infinity of the continuous line between the integers, creating an infinity of computational exceptions with a whole class of forbidden numbers.

This brings us to floats.

---

3. Float environment (Q)

A learned reader already anticipates the answer to this conundrum. But we should still go through with all the motions available to our moveset.

We can try to measure the intervals between integers. The problem remains the same - an interval between two integers can't be measured discretely.

So we reach for the operations we just acquired: ×a and ÷a. These are the operations that broke the integer line's perfection. Maybe they can be used to amend the state of affairs.

In §2, ÷a was introduced as the operation of partitioning. Geometrically, it does something more precise: it **bends the line**. From the POV of the integer line it is recorded as a strange occurrence: the line bends out of sight and a part of the whole line is concealed. The inversion point marks the site of the bend. This creates an overflow event where the state of the previous environment: one anchor point = one inversion point is no longer true, and we have more unaccounted inversion points than the anchor points can track.

In response to this problem we can use ×a as an unbending operation, that returns the line to its flat recognizable shape. This doesn't make the inversion point disappear, but records it as the address of the bend. We operate on the premise: "if it can be traced – it can be operated on later".

We know the recorded trace of this as a ratio a_1/a_2. It is not a point on the line. The ratio tells us that ÷a was applied, and where, and that ×a undid it. It creates a computational bug-fix of the operation.

Once we repeat the available operations: ×a and ÷a - ad infinitum, in a strict combinatorial fashion (not to import a choice function) we can learn the following:

Take the interval between two adjacent integers. Apply ÷a. The interval is partitioned. Apply ÷a again. Each part is partitioned in turn. Repetitive ÷a produces an infinite amount of partitions. Each partition nears one integer, without terminating.

Note that previously we were limited to an operation/result computation, but binary arrays allow us to sequence computations without resolving them, this is the demonstration of the power our little construct.

So, then we add all the products of this partition.

This sum can be represented with no metric – because we already know how to mark infinities. The sum is the whole: the infinite partitions toward another integer.

Taken together – they form the boundaries of the interval as a line between decomposition and composite sum. This is obvious to us, but we had to use the grammar iteratively to express the simple truth: there is an infinity on both ends of the interval, proving beyond any doubt that the intervals can't be discretely measured.

This is not nothing. It gives a possibility to reframe the problem we are dealing with.

We are not looking for points between integers to measure discretely. We are looking for the **geometry that bending produces**.

3.1 Inventorizing

To go further, we have to make a step back and recognize how our ruleset has changed:

A. Direction

Our grammar supports **direction** now.

For the continuous line we had +/- directions of the line stretches/constricts, marked by an inversion point – null.

For the mint-condition integer line +/- could be recognized as directions relative to the null. But the advance to float environment produced an uncountably infinite amount of inversion points. Notice that we don’t have a single null to distinguish the + anchor points and – anchor points.

B. Nulls and cycles

Every environment possesses a boundary defined as null:

1) Binary: Null of the unmarked (precedes the bit) and null of the terminated (supersedes binary environment).

2) Integer: Null of the inversion (point where constricting changes to stretching) and null of the infinity (a boundary where counting becomes transfinite)

3) Float: Minimal infinity and maximal infinity - the two terminal nulls at the ends of any interval.

It seems that each environment is "wrapped-up" between two nulls. An alpha and omega condition if you will. So, from this we can deduce that each environment is a **"cycle".** Null is the boundary for each environment and each computational instance we run, be it binary, integer or float is a cycle. And geometrically this implies a **Loop**.

C. Order

All of the previous inventory provides us with all the grammar needed to formulate a decision on how to answer the overflow state we found ourselves in: *inversion points > anchor points.*

Firstly, we can recognize that we were operating on a triadic scale of object, structure, instance ("osi"). We already recognized that each instance was a cycle. So, we can create a filtering operation by assigning each environment to be its own scale.

This way we can loop the intervals as cycles and render the integers as a binary string. The specialist may want to name the topology of this action, but we have to follow the incremental development rule of the game.

We did not invent a **canonical order of the integers** yet! Counting was an operation that produced a count, but ordering is a higher-class operation that let's us call the counts as addresses. That's why the easiest solution is to use ×a and ÷a iteratively to learn that there is an already existing order of odd/even, described through operations.

Even is when the division is even and points either to an anchor point or a ratio. (A pun: it gets even with the demarcation)

Odd is when division produces a result that is not a ratio. (A pun: which is definitely odd, because we did not compile the float as a data structure yet)

Even follows odd, odd follows even. This is a rule that we will utilize to the fullest.

D. Local direction

Using the canonical order of the integers we can abstract it one level higher and operationalize it through [n, n+1, n+2, …] and [n, n-1, n-2, …] using odd/even as operators for our operations. But it is still relative from one anchor point to another – with boundaries being local.

We don't have a single n to unite them all. We don't have **an origin**.

3.2. The origin and the float

Let's define an origin through a commonality that all environments possess.

Face	**Alpha null**	**Omega null**
Binary	Unmarked	terminated
Integer	Inversion	infinity
Float	min-infinity	max-infinity

This commonality is a null. We can recognize that a null is a mark for non-existence and non-completion. Thus we can extract a null as a *label* and introduce it into our moveset:

×0 operation as an operation of unmarking any object in any environment.

There is nothing mystical about it – it just returns the marked object with a label for filtering between two modes of existence that we could operate on. We can't operate on the instances yet, but we can distinguish two operational levels of complexity:

A. Grammar layer

Object: Binary flag for marked/unmarked

Structure: Integer string of counters/directions

Instance: A sequence of cycles /orders

B. Structure layer

Instance: Binary flag for booting/resetting instance

Structure: Ordered sequence of odd/even numbers

Object: Continuous interval between integers

We can think of this abstraction layer as a runtime between object/structure organized like a bi-sheet. Like a CPU that computes and GPU that renders a computational object. (remember that one of the three interfaces was informatical, and this is the layer that it shines on).

Having the runtime accounted for – we can use the newly acquired operation of ×0 to exclude any exception out of a runtime, thus creating some kind of sub-space or a collecting bin.

It's also where we will put an ÷0 operation, because it is unfathomable, so a perfect first candidate for testing if bin works. It does wonders.

Let's apply ×0 at every scale.

- **Binary**, ×0 on a string of marks: 1111 → 0000, labeled inactive.
- **Integer**, ×0 on a counter: n → 0, labeled inactive.

Effectively this passes everything through a filter that can be used as an origin "0".

On a binary level: 0 of no-flag; 0 of no string; 0 of no existence.

On an integer level: 0 of no counters (anchors); 0 of no inversion; 0 of no continuity.

The origin is a point to which every environment, structure and instance can be reduced to (i.e. labeled).

The line then is re-factored to have a +/- filter relative to the 0 as origin as an ordering tool of a global scale (alongside the odd/even which is still useful, but on a more local scale)

And as such we find the ultimate anchor point that we can perform the ordering against, thereby establishing the float environment:

Structure: Integer string.

With canonical direction of +/- and bounded by infinity of counting in either direction. A relative measure.

A sequence of anchors and intervals, conjoined. +b/−b operate on the anchors. ×b/÷b operate on composition. ÷b bends; ×b unbends. This is the same unbending action as in the integer environment; the environment has changed, not the operation.

Object: Interval cycle.

We can take the boundaries of an interval and cycle them by flagging the boundaries with a 0 that stands for the proximity to an integer. This bears a structure of a ring.

Instance: Binary sequence of string/cycles.

We render it as a **float** – an interface for the otherwise unoperable instance:

A float, written a.bbbbb. The integer part a is the anchor address - which cycle you are on. The fractional part bbbbb is the cycle position - where on the cycle.

The ×0 operation works on a float in the same fashion as on integer – and thus we have established a computational architecture that supports different levels of abstraction and runtime environments.

The whole construction has developed from a single dot with a minimal moveset into a (3 scales) × (2 sheets) tensor. This is a substantial upgrade from an array of the integer environment that allows for cross-level operation.

And if the system can now do a cross-level operation internally, with everything established, we can advance our moveset once more. What a float environment can try to achieve is a dimensional-lift operation to try and reach the complex plane.

3.3 The hubris operation

Repeated ×b and ÷b produce an emergent operation of ^b exponentiation and its inversion √b root.

These are very special operations that unlock the full potential of (Q) and they are a perfect solution for the inversion point overflow that we caused not so long ago.

Firstly, we unlock the power of powers and roots. Powers let us find every discrete integer that doesn't produce a rogue inversion point, establishing a respectable class of numbers. Then √b lets us address every inversion point on an interval through an irrational number! And after we address it with a √b we can lift the number back as an anchor point through ^b operation!

So, no more unknown, everything is marked, which means for every existing point an anchor has been made as a permanent address. The system flourishes in computational power and can enjoy every perk that we've unlocked so far.

That's when we notice. ^b governs the dimensional lift. And if it is in our moveset – then it can be added into our grammar to work on a structural level:

- ^1 produces string structure.
- ^2 produces array structure.
- ^3 produces tensor structure.

These are all already existing structural dimensions. ^4 would produce a four-index tensor. What would happen if our structure flips that switch? So, we do.

Once the system flips that switch it has to notice a simple fact about the float environment that it was only computing transfinitely.

The infinities are truly infinite. The moment we develop a √b operation – we are forever bound to use it ad infinitum to mark every possible address – we hit the wall of the transfinite counting once again.

Before this operation existed we could only guess what a repeated use of ÷b does to a line. It produces a curve, and our system is stuck computing a transfinite smoothing of a curve for all eternity with no gap left uncalculated.

Then the structure would flip ^5; ^6; ^7 etc and quickly transition into the fourth environment – complex field.

The hubris operation creates a split between a real and complex parts of a number, where a real number is computed through binary/integer/float environments and then the float environment bleeds half-way into a transfinite curvature.

3.4. Tending to the wound

Now that we know what a float environment looks like we can once again define its operations on its own terms and a different scale (structural:

Cyclical shift - +b and −b. The first pair acts within a single ring. The ring inherits its orientation from §3.1: the interval's two bounding anchors are one odd, one even, and the canonical direction is odd → even. +b shifts the position by one step in the canonical direction; −b shifts it by one step against. The two are inverses - applying one after the other returns the position to where it began - so they are distinguishable by what they do.

Modulo multiplication - *b**.**
The second operation *composes* shifts within one ring. Applying +b several times, or shifting by several rotations, is a single act of modulo multiplication. *b moves several times around the same gap in one operation, without leaving the ring.

Modulo division - /b**.**
The third operation *distributes* the ring across cycles. /b assigns each point of the ring a remainder, fracturing the ring into its cycle structure. This is what

produces the periodic representation of the interval - what was one ring becomes a periodic sequence of positions, each tagged by its remainder.

Each operation acts at three scales, following the grammar of Section 1:

- **Object scale** - a single point on a single ring.
- **Structure scale** - a collection of points on a single ring, or a single point across many rings.
- **Instance scale** - the whole sequence, operated on as a single float instance.

Then we have transcendent float environment operations that operate on a complex plane.

- √b: **root extraction**. Send the number into a complex plane.
- ^b: **exponentiation**. Expatriate the number from a complex plane.

But under the logic of this construction it is not a free bus-ride.

If we take a number "b" and send it into higher dimensions with b^4, let's define b as b = 2. We receive 16, we successfully compounded the number 2 into 16. It is a resounding success for dimensional travel.

But what if we have an expression 16 = b^4. We define b = |2|. That's because after we sent the number b into higher dimensions – we no longer know if it has the same +/- orientation on our real number line. In other words, once we send a number into a higher dimension it may undergo some form of change, that is not renderable in our float environment.

Operation |b| (**magnitude extraction**) is a protection we use to catch complex number imposters and to separate them from our "safe" counters and addresses.

Same applies to the other operation.

If we take a number "b" and send it into a complex plane with √b, let's define b = 4. We receive a √4. Once we return the number back we get:

$$\sqrt{4} = 2$$

But we've sent this number into a complex plane. How can we prove without defining that it is the same number and doesn't possess any kind of information that the float environment can't support as a readable data? (e.g. phase). The grammar seems to support such possibility in the context of this thought experiment. As well as an inverse operation of |b|.

Let's name it [[b]] – phase obfuscation. An operation that marks the number as probably possessing some kind of phase information that the float environment can't render.

√4 = [[2]]

And this is where our thought experiment ends, because our construction has touched the complex plane and come back.

---

4. Conclusion

Let's look back at the journey. We began with a single bit and a two-move grammar and ended with a curve and a fractured number line.

We have traversed multiple computational environments and witnessed that every environment developed through handling some kind of overflow and in the process – created another, bigger overflow state.

Another point to make is that no environment we investigated could handle operations on its own instance. And when the float environment attempted to – it fractured into an infinity of exceptions. This can be thought of as the generative grammar equivalent of Gödel's incompleteness condition: **a system cannot perfectly operate on the grammar that enables its own existence.** And a possible clause: *unless the transition fee is recognized and paid in advance.* E.g. the system loses its coherence as a system or other conditions.

The binary geometric operand automata that we ran was instanced over three interfaces, and while the main focus of this paper was focused on the mathematical and informatical facets – the geometric facet was left unreflected.

The Geometric ladder of operational freedoms		
Algebraic Operation	**Geometric Freedom**	**Geometric Object**
+1; -1	Data structure (string)	Point
+a; -a	Continuity	Line
×a; ÷a	Orthogonality	L-shape
+b; -b	Connectivity	Grids
×b; ÷b	Polygon	Figures
^b; √b; |b|; [[b]]	Curvature	Apeirogon

We can interpret the geometric facet of this thought experiment to pose a modest amount of concerning conclusions. If we read this table as a spatial history, it looks like a march from one dimension to two. But if we read it strictly as the *shadow of the moveset*, the conclusions are far stranger.

1. We have not defined an embedding for this thought experiment.
Because we never established an external coordinate space to place our dot inside, we cannot actually claim to have entered a second dimension. The geometry here is strictly a shadow of the algebraic rules.

2. A point can be rendered as a string.
We begin with a 0D object that has no coordinates, no extent. But we can render it as a 1D informational object, that using binary operations of +1; -1 can first afford to manipulate its own existence and then "layer" itself sequentially into a "string": a data structure with a marked internal storage. Under the condition of this thought experiment – a point can be "denser".

3. The L-shape is not the birth of the second dimension.
When we introduce ×a; ÷a, the algebra demands fractions. The geometry responds with an L-shape. But we have not defined a second axis here. So, effectively what happens is that division obfuscates parts of a line. When you divide 3 by 2, you see 1.5, but the half of the line that created the fraction is folded out of sight into an unknown dimension. And we can only recognize that we formed an L-shape, from a higher-dimensional representation.

4. Grids are not spaces.
The +b; -b operations introduce cyclical connectivity, and the geometry responds with grids. But a grid is not a 2D plane. It is a collection of intersecting 1D paths. Even if you run a Peano curve through it, it possesses no actual area - only the illusion of a surface made from orthogonally crossing lines.

5. Polygons create figures, not space.
With ×b; ÷b, the grid closes on itself, creating polygons. But again, the geometry is deceiving. A polygon creates a *figure* - a closed boundary - not a spatial interior. It is a 1D loop that has learned how to contain itself without ever becoming a surface.

6. The Apeirogon is the Gödelian threshold.
Finally, the operations ^b; √b demand a dimensional lift to create a smooth curve. Because the float environment lacks the complex plane to render it, it becomes stuck in a transfinite smoothing. The resulting geometric object is an Apeirogon - a polygon with a countably infinite number of straight sides.

The Apeirogon is not a failure. It is exactly what the Overflow looks like geometrically. It is the shape of the system hitting its Gödelian threshold. When the algebra can no longer hide its exceptions in folded L-shapes or closed figures, it shatters into an infinite regress of straight lines, leaving a fractal scar that only looks like a curve from a distance.

7. We may have actually not crossed from 1D to 2D.

By the end of the journey, we have not stepped into a 2D space. Every "new" geometric object is just the current 1D environment fracturing, folding, or hiding its algebraic remainders. And the way that the geometry enables it is with tricks of “smoke and mirrors” such as obfuscation.

---

5. Questions for the Reader

1. Does Shor’s algorithm work the way it does because there is no distinct boundary between real and complex numbers?
In this experiment, the float environment `Q` did not cleanly end where the complex plane `C` began. It bled into it. The operation `√b` sent a number into the complex plane, and `^b` brought it back carrying an unrendered phase. If the boundary between real and complex is not a wall, but a transfinite smoothing - a continuous curve - then quantum algorithms that leverage phase and periodicity might work precisely because they operate in that "bleed," exploiting the underlying continuity of the algebraic structure rather than treating real and complex as separate domains.

2. Does ÷0 in the bin do any further work?
Earlier in the experiment, we created a runtime filter (`×0`) and swept the unfathomable `÷0` into it. We labeled it "inactive" and moved on. But if `×0` acts as an origin filter, what does it mean to divide by that origin?

3. Why the environments follow a 2-4-8 progression of operations?

The binary environment (N) gave us 2 operations (`+1`, `-1`). The integer environment (Z) gave us 4 (`+a`, `-a`, `×a`, `÷a`). The float environment (Q) gave us 8 (`+b`, `-b`, `×b`, `÷b`, `^b`, `√b`, `|b|`, `[[b]]`). We recognized it as such, but this still seems like a strange coincidence that the progression of algebraic operations can at all be rendered with powers of a 2. What do you think?

4. Do you see any benefit in the concept of obfuscation?

It may lead to new potential discoveries.

5. Was the complex plane actually breached in this paper?

It was under the constraints of a generative grammar introduced in the larger monograph.

6. Is the grammar minimal in the relevant sense?
Every operation in the moveset was introduced as a forced response to an overflow. But forcedness is a claim, not a guarantee. Can any operation be removed, or derived from the others, without collapsing the ladder? And

separately: can the same ladder be generated with fewer interpretive affordances - fewer labels, fewer geometric renderings - or does each label do irreducible work? I would like to see it optimized by a capable specialist.

Learn more about these and other questions in the full monograph.

Source boundary:
This paper is a standalone synthesis of the schema design and generative architecture presented in Kirill I. Kovlekov, *Poetica Mathematica: Cogitatio ex Nihilo et Speculum Scientiae* (Version 0.25). The framework of the three operational faces (informatic, geometric, algebraic), the progression of the moveset, and the geometric ladder of operational freedoms follow the monograph's internal sequence, chiefly the Triptych Engine extrinsic architecture in Chapter III (§§3.3.1–3.3.3, covering the algebraic, geometric, and spectral axes) and the exploratory intrinsic L-meta note (PDF pp. 1100–1104). The concept of the dot as a binary geometric operand, the derivation of the Float environment, and the identification of the Apeirogon as the Gödelian threshold of the orthogonal grammar are reconstructed here as a self-contained thought experiment. The geometric mapping is presented as a structural shadow of the algebraic moveset, not as a claim of physical embedding or standard spatial dimension. The full monograph is available via Zenodo at https://doi.org/10.5281/zenodo.23698076.